

Неделя 1

Базовые понятия обучения с подкреплением

Обучение с подкреплением — это метод машинного обучения, при котором алгоритм (агент) учится принимать оптимальные решения, взаимодействуя с окружающей средой методом проб и ошибок. Агент получает награды или штрафы за свои действия, стремясь максимизировать суммарное вознаграждение.

Процесс строится на взаимодействии двух сущностей: агента и окружающей среды:

1. Агент (agent) – это алгоритм или сущность, которая принимает решения и совершает действия.
2. Среда (environment) – это внешний мир или система, в которой действует агент. Он ничего не знает о ней, а только получает новые состояния и награды в ответ на действия. Взаимодействие между агентом и средой происходит в циклическом режиме:
3. Действие (Action, a): агент совершает шаг (например, показывает баннер пользователю).
4. Состояние (State, s): среда сообщает агенту информацию о текущей ситуации (например, характеристики пользователя в интернете). Важно, что вероятность перехода в следующее состояние зависит только от текущего состояния и действия, а не от всей предыдущей истории (Марковское предположение).
5. Награда (Reward, r): среда дает числовую обратную связь, оценивающую качество последнего действия (например, кликнул ли пользователь на баннер). Улучшение стратегии (Policy, $\pi(a|s)$): правило, по которому выбирается действие в конкретном состоянии

Метод кросс-энтропии

Метод кросс-энтропии (Crossentropy method, CEM) – метод стохастической оптимизации (эволюционный алгоритм), который используется для поиска оптимального решения в задачах с редкими наградами, в том числе в обучении с подкреплением.

Табличный метод кросс-энтропии

Этот подход применяется в задачах с небольшим количеством состояний и действий. Стратегия $\pi(a, s)$ описывается как матрица вероятностей следующего вида:

$$\pi(a, s) = A_{s,a}$$

$$E = \{(a_0, s_0), \dots, (a_k, s_k)\}$$

$$\pi(a, s) = \frac{\sum_{s_t, a_t \in E} [s_t = s][a_t = a]}{\sum_{s_t, a_t \in E} [s_t = s]}$$

Нейросетевой метод кросс-энтропии

Идея состоит в том, чтобы использовать нейронную сеть для аппроксимации стратегии. На каждом шаге действия выбирается с помощью предсказания сети. Сеть максимизирует правдоподобие из элитных сессий $\pi = \arg \max \sum_{s_i, a_i \in E} \log \pi(a_i, s_i)$.

Неделя 2

$V(s), Q(s, a)$

$V(s)$ – функция стоимости состояния (state-value function) – это математическое ожидание суммарного дохода, который агент может получить, начиная с состояния s и далее следуя своей политике π .

$$v_{\pi}(s) = E_{\pi}[G_t | S_t = s] = \sum_a \pi(a|s) \sum_{r, s'} p(r, s' | s, a) [r + \gamma v_{\pi}(s')] \quad (1)$$

Интуитивный смысл $V(s)$ заключается в том насколько выгодно для агента находиться в данном состоянии s , если он будет продолжать действовать согласно своей текущей политике π . Считается, что одна стратегия лучше другой, если её функция $V(s)$ выше или равна для всех возможных состояний среды

$Q(s, a)$ – функция состояния и действия (action-value function) – это ожидаемое вознаграждение, которое агент получит, если в состоянии s он сначала совершит конкретное действие a , а во всех последующих состояниях будет придерживаться политики π .

$$q_{\pi}(s, a) = E_{\pi}[G_t | S_t = s, A_t = a] = \sum_{r, s'} p(r, s' | s, a) [r + \gamma v_{\pi}(s')] \quad (2)$$

Метод policy iteration

Метод итерации стратегии (Policy Iteration) — это алгоритм, предназначенный для поиска оптимальной стратегии в Марковских процессах принятия решений (MDP). Он представляет собой циклическое чередование двух основных этапов: оценки текущей стратегии и ее последующего улучшения.

Начинаем с произвольной стратегии π и произвольной $V(s)$. Итеративно обновляем ценность каждого состояния, используя уравнение Беллмана:

$$V(s) = \sum_{s', r} p(s', r | s, \pi(s)) [r + \gamma V(s')] \quad (3)$$

Если новая стратегия совпадает со старой, то алгоритм завершается, иначе возвращаемся к уравнению Беллмана, но с новой стратегией:

$$\pi(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')] \quad (4)$$

Неделя 3

Q-learning, Sarsa

Q-learning и SARSA – это два алгоритма, относящиеся к семейству методов временных различий (Temporal Difference, TD-learning) и решающие одну задачу: учат агента находить оптимальную стратегию поведения в среде без знания ее математической модели (model-free). Ключевая особенность Q-learning заключается в том, что функция ценности состояний и действий $Q(s, a)$ обновляется независимо от того, какую стратегию (политику) агент использует для выбора следующего

действия. В отличие от Q-learning, SARSA (State-Action-Reward- State-Action) оценивает и улучшает ту же самую стратегию, по которой агент реально осуществляет выбор действий в среде. Формулы обновлений:

$$Q(s, a) = (1 - \alpha)Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a'))$$

$$Q(s, a) = (1 - \alpha)Q(s, a) + \alpha(r + \gamma EQ(s', a'))$$

Exploration vs Exploitation

Дилемма Exploration vs Exploitation заключается в поиске оптимального баланса между получением новых знаний о среде и использованием уже накопленного опыта для максимизации награды. Exploitation – выбор агентом действий, которые на основе текущих оценок кажутся наиболее выгодными. Цель заключается в максимизации немедленного выигрыша в краткосрочной перспективе.

Exploration – выбор агентом действий, отличных от текущих оптимальных, с целью сбора новой информации о среде и проверки альтернативных стратегий. Цель заключается в поиске глобального оптимума и максимизации суммарной награды в долгосрочной перспективе.

Основных методов решения дилеммы 2:

1. ϵ -greedy стратегия. С вероятностью ϵ выбирается действие случайное a , далее значение ϵ уменьшается.

2. softmax стратегия. Выбирается действие исходя из функции $\pi(a, s) = \frac{e^{\frac{Q(s,a)}{T}}}{\sum_{a_i} \frac{Q(s,a_i)}{T}}$

Неделя 4

Deep Q-Network

DQN - алгоритм, объединяющий классический Q-learning с глубокими сетями. В классическом алгоритме ценность каждого действия записывается в таблицу $Q(s, a)$, но если задача сложная, то таблица становится очень большой. Поэтому идея состоит в том, чтобы заменить ее сетью, которая на основе s дает предсказание для всех возможных Q -значений. Базовая архитектура DQN для обработки визуальных состояний обычно состоит из backbone части из сверточных слоев, после которых идут полносвязные, выдающие результат.

Experience replay

Experience replay - техника, позволяющая агенту эффективно использовать накопленные данные для обучения нейронной сети. Агент сохраняет прошлые взаимодействия (s, a, r, s') , из которых он в процессе тренировки выбирает случайные подвыборки и обновляет веса на их основе. Данный подход позволяет решить проблему нарушения независимости данных и проблему с "забыванием" части среды.

Target network

Target network – это вспомогательная нейронная сеть, используемая в алгоритмах глубокого обучения для стабилизации процесса обучения. Идея состоит в том, чтобы использовать сеть-backbone с

замороженными весами для вычисления целевого значения, причем loss-функция будет выглядеть следующим образом:

$$L(\theta) = E_{s,a,r,s' \in Buffer} ((Q(s, a, \theta) - (r + \gamma \max_{a'} Q(s', a', \theta^{frozen})))^2) \quad (5),$$

где θ, θ^{frozen} обновляемые и замороженные веса соответственно.

Double DQN

Double DQN – это усовершенствованная модификация алгоритма DQN. Ее главная цель – устранить систематическую проблему завышения оценок истинной полезности действий, которой страдает классический DQN. DQN начинает верить, что определенные действия принесут гораздо больше награды из-за условия:

$$E[\max\{\xi_1, \xi_2, \dots, \xi_n\}] \geq \max\{E\xi_1, E\xi_2, \dots, E\xi_n\} \quad (6),$$

где ξ_k - случайная величина. Для расчета целевого значения DDQN используется формула

$$r + \gamma Q(s', \arg \max_{a'} Q(s', a', \theta), \theta^{frozen}) \quad (7)$$